

Content Provider In Android:

A content provider manages access to a central repository of data. You implement a provider as one or more classes in an Android application, along with elements in the manifest file. One of your classes implements a subclass `ContentProvider`, which is the interface between your provider and other applications. Although content providers are meant to make data available to other applications, you may of course have activities in your application that allow the user to query and modify the data managed by your provider.

Before you start building a provider, consider the following:

Decide if you need a content provider. You need to build a content provider if you want to provide one or more of the following features:

You want to offer complex data or files to other applications.

You want to allow users to copy complex data from your app into other apps.

You want to provide custom search suggestions using the search framework.

You want to expose your application data to widgets.

You want to implement the `AbstractThreadedSyncAdapter`, `CursorAdapter`, or `CursorLoader` classes.

You don't need a provider to use databases or other types of persistent storage if the use is entirely within your own application and you don't need any of the features listed above. Instead, you can use one of the storage systems described on the [Saving App Data](#) page.

If you haven't done so already, read the topic [Content provider basics](#) to learn more about providers and how they work.

A content URI is a **URI that identifies data in a provider**. Content URIs include the symbolic name of the entire provider (its authority) and a name that points to a table (a path). When you call a client method to access a table in a provider, the content URI for the table is one of the arguments

Content URI patterns

To help you choose which action to take for an incoming content URI, the provider API includes the convenience class [Uri Matcher](#), which maps content URI "patterns" to integer values. You can use the integer values in a `switch` statement that chooses the desired action for the content URI or URIs that match a particular pattern.

```
public class ExampleProvider extends ContentProvider {
    ...
    // Creates a UriMatcher object.
    private static final UriMatcher uriMatcher = new
    UriMatcher(UriMatcher.NO_MATCH);

    static {
        /*
         * The calls to addURI() go here, for all of the content URI patterns that
         the provider
         * should recognize. For this snippet, only the calls for table 3 are
         shown.
         */

        /*
         * Sets the integer value for multiple rows in table 3 to 1. Notice that no
         wildcard is used
         * in the path
         */
        uriMatcher.addURI("com.example.app.provider", "table3", 1);

        /*
         * Sets the code for a single row to 2. In this case, the "#" wildcard is
         * used. "content://com.example.app.provider/table3/3" matches, but
```

```

        * "content://com.example.app.provider/table3 doesn't.
        */
        uriMatcher.addURI("com.example.app.provider", "table3/#", 2);
    }
...
    // Implements ContentProvider.query()
    public Cursor query(
        Uri uri,
        String[] projection,
        String selection,
        String[] selectionArgs,
        String sortOrder) {
...
        /*
        * Choose the table to query and a sort order based on the code
returned for the incoming
        * URI. Here, too, only the statements for table 3 are shown.
        */
        switch (uriMatcher.match(uri)) {

            // If the incoming URI was for all of table3
            case 1:

                if (TextUtils.isEmpty(sortOrder)) sortOrder = "_ID ASC";
                break;

            // If the incoming URI was for a single row
            case 2:

                /*
                * Because this URI was for a single row, the _ID value part is
                * present. Get the last path segment from the URI; this is the _ID
value.
                * Then, append the value to the WHERE clause for the query
                */
                selection = selection + "_ID = " + uri.getLastPathSegment();
                break;

```

default:

```
...  
    // If the URI is not recognized, you should do some error handling  
here.  
}  
    // call the code to actually do the query  
}
```